

Unidad 3, creando Front + Back + DB (MySQL)

1. Estructura general del proyecto

Para esta etapa del curso (Unidad 3), se trabaja con **dos proyectos separados**:

Proyecto	Descripción	Puerto por defecto
Backend (Spring Boot)	API REST con acceso a base de datos MySQL.	8080
Frontend (React)	Interfaz web (SPA) que consume la API REST.	3000

Ambos deben ejecutarse **en simultáneo**:
Spring Boot actúa como **servidor** (provee datos y servicios), mientras React actúa como **cliente** (consume los endpoints REST).

2. Proyecto Backend – Spring Boot + MySQL

a) Configuración inicial

1. Crear proyecto en [Spring Initializr](#) con:
 - **Spring Web**
 - **Spring Data JPA**
 - **MySQL Driver**
 - (Opcional) Spring Security + JWT
 - (Opcional) Swagger (`springfox-boot-starter`)
2. En el archivo `pom.xml`, asegurar que están las dependencias de **JPA, MySQL y Swagger** (si se usa documentación REST).

b) Configuración MySQL (`application.properties`)

```
spring.datasource.url=jdbc:mysql://localhost:3306/nombre_bd?useSSL=false&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=tu_password
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
```

 Si se usa *MySQL Workbench*, crear previamente la base `nombre_bd` antes de ejecutar el proyecto.

c) Estructura básica del backend

```
src/main/java/com/example/backend/
├── model/           → Clases @Entity (tablas)
├── repository/      → Interfaces JpaRepository
├── service/         → Lógica de negocio
├── controller/     → @RestController (API REST)
└── config/         → Configuración CORS, Swagger, etc.
```

Ejemplo de API

```

@RestController
@RequestMapping("/api/products")
@CrossOrigin(origins = "http://localhost:3000")
public class ProductController {
    @Autowired
    private ProductService service;

    @GetMapping
    public List<Product> list() { return service.getAll(); }

    @PostMapping
    public Product save(@RequestBody Product p) { return service.save(p); }
}

```

d) Encendido del backend

Desde consola o IDE:

```
mvn spring-boot:run
```

Comprobación:

- Swagger UI → <http://localhost:8080/swagger-ui/>
 - Base de datos → revisar en MySQL Workbench que se estén creando las tablas.
-



3. Proyecto Frontend – React (cliente)

a) Crear proyecto

```

npx create-react-app frontend
cd frontend
npm install axios react-router-dom

```

b) Estructura básica

```

src/
├── pages/           → Vistas (Home, About, Admin, etc.)
├── components/      → Componentes reutilizables
└── services/        → Módulos para llamadas HTTP (axios)

```

c) Ejemplo de servicio Axios

```

// src/services/ProductService.js
import axios from "axios";
const BASE_URL = "http://localhost:8080/api/products";

export const getAllProducts = () => axios.get(BASE_URL);
export const createProduct = (p) => axios.post(BASE_URL, p);

```

d) Ejemplo de uso en componente

```

import { useEffect, useState } from "react";
import { getAllProducts } from "../services/ProductService";

export default function ProductList() {
    const [products, setProducts] = useState([]);

    useEffect(() => {
        getAllProducts()
            .then(res => setProducts(res.data))
            .catch(err => console.error(err));
    }, []);
}

```

```
return (
  <div className="container my-4">
    <h2>Lista de productos</h2>
    <ul>
      {products.map(p => (
        <li key={p.id}>{p.name} - ${p.price}</li>
      ))}
    </ul>
  </div>
);
}
```

🔗 4. Conexión entre React y Spring Boot

Comunicación

- React realiza peticiones HTTP (GET, POST, PUT, DELETE) usando **axios** o **fetch**.
- Spring Boot responde con datos JSON.
- Ambos deben estar levantados:
 - React en `http://localhost:3000`
 - Spring Boot en `http://localhost:8080`

CORS

Se debe permitir la comunicación cruzada:

```
@CrossOrigin(origins = "http://localhost:3000")
```

o globalmente con:

```
@Configuration
```

```
public class CorsConfig implements WebMvcConfigurer {
  @Override
  public void addCorsMappings(CorsRegistry registry) {
    registry.addMapping("/api/**")
      .allowedOrigins("http://localhost:3000")
      .allowedMethods("GET", "POST", "PUT", "DELETE");
  }
}
```

⚡ 5. Flujo de ejecución completo

Paso	Acción	Descripción
1	Iniciar MySQL	Abrir servidor local o XAMPP.
2	Crear BD vacía	Ejemplo: <code>CREATE DATABASE tienda;</code>
3	Iniciar backend	Ejecutar Spring Boot (<code>mvn spring-boot:run</code>) → genera tablas.
4	Probar API	Desde Swagger o Postman (<code>http://localhost:8080/api/...</code>).
5	Iniciar frontend	<code>npm start</code> → interfaz React visible en <code>localhost:3000</code> .
6	Interactuar	Crear, editar o listar datos → cambios reflejados en MySQL.

✅ Resultado final esperado

- Backend (Spring Boot):
 - Endpoints REST funcionales (CRUD).
 - Conectado a MySQL.
 - Accesible en `http://localhost:8080/api/....`
- Frontend (React):
 - Consume la API correctamente.
 - Permite visualizar y modificar datos reales.
 - Sin errores de CORS.

proyecto Spring Boot (backend) y MySQL

✿ Paso 0 – Preparar MySQL

1. Levantar MySQL (XAMPP, MySQL Server, Docker, etc.).
2. Crear la base de datos (puede ser por Workbench o consola):

```
CREATE DATABASE tienda CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

3. Tener claro:
 - o usuario (ej: root)
 - o password (ej: root o la que usen siempre)

🔨 Paso 1 – Crear proyecto Spring Boot (backend)

1. Ir a <https://start.spring.io/>
2. Configurar:
 - o Project: **Maven**
 - o Language: **Java**
 - o Spring Boot: versión estable actual
 - o Group: `com.example`
 - o Artifact: `backend` (o el nombre que prefieras)
3. En **Dependencies**, agregar:
 - o **Spring Web**
 - o **Spring Data JPA**
 - o **MySQL Driver**
 - o (Opcional pero recomendado) **Spring Security**, **springdoc-openapi** o **springfox-boot-starter** para Swagger.
4. Click en **Generate**, descargar y descomprimir.
5. Abrir el proyecto en tu IDE (IntelliJ, Eclipse, VS Code).

⚙️ Paso 2 – Configurar conexión a MySQL

En `src/main/resources/application.properties` (o `application.yml` si prefieres) configura la conexión:

```
spring.datasource.url=jdbc:mysql://localhost:3306/tienda?useSSL=false&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect

# Opcional: para ver mejor los logs SQL
logging.level.org.hibernate.SQL=DEBUG
logging.level.org.hibernate.type.descriptor.sql.BasicBinder=TRACE

# Desactivar seguridad en endpoints para desarrollo
spring.security.user.name=user
spring.security.user.password=1234

# Permitir acceso libre a Swagger y a la API
spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.security.servlet.SecurityAutoConfiguration
```



Paso 3 – Crear la entidad (modelo JPA)

Ejemplo: Product (cada estudiante la adapta a su dominio).

```
// src/main/java/com/example/backend/model/Product.java
package com.example.backend.model;

import jakarta.persistence.*; // o javax.persistence.* según versión

@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY) // usa AUTO_INCREMENT en MySQL
    private Long id;

    @Column(nullable = false, length = 120)
    private String name;

    private String category;

    private Double price;

    // Constructor vacío obligatorio para JPA
    public Product() {}

    // Constructor opcional
    public Product(String name, String category, Double price) {
        this.name = name;
        this.category = category;
        this.price = price;
    }

    // Getters y setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getCategory() { return category; }
    public void setCategory(String category) { this.category = category; }

    public Double getPrice() { return price; }
    public void setPrice(Double price) { this.price = price; }
}
```

Este patrón es exactamente el que se propone en las guías para entidades tipo Book/Producto.



Paso 4 – Crear el repositorio (DAO con JPA)

```
// src/main/java/com/example/backend/repository/ProductRepository.java
package com.example.backend.repository;

import com.example.backend.model.Product;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ProductRepository extends JpaRepository<Product, Long> {
    // Si necesitas búsquedas específicas, puedes agregar métodos aquí
}
```

Esto sigue el patrón `JpaRepository` que explican en las PPT: repositorios para operaciones CRUD.



Paso 5 – Crear el servicio (lógica de negocio)

```
// src/main/java/com/example/backend/service/ProductService.java
package com.example.backend.service;
```

```
import com.example.backend.model.Product;
import com.example.backend.repository.ProductRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
```

```
import java.util.List;
```

```
@Service
public class ProductService {

    @Autowired
    private ProductRepository repo;

    public List<Product> getAll() {
        return repo.findAll();
    }

    public Product getById(Long id) {
        return repo.findById(id).orElse(null);
    }

    public Product create(Product product) {
        return repo.save(product);
    }

    public Product update(Long id, Product product) {
        Product existing = getById(id);
        if (existing == null) return null;
        existing.setName(product.getName());
        existing.setCategory(product.getCategory());
        existing.setPrice(product.getPrice());
        return repo.save(existing);
    }

    public void delete(Long id) {
        repo.deleteById(id);
    }
}
```



Paso 6 – Crear el controlador REST (endpoints)

```
// src/main/java/com/example/backend/controller/ProductController.java
package com.example.backend.controller;
```

```
import com.example.backend.model.Product;
import com.example.backend.service.ProductService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
```

```
import java.util.List;
```

```
@RestController
@RequestMapping("/api/v1/products")
@CrossOrigin(origins = "http://localhost:3000") // permite React en localhost:3000
public class ProductController {

    @Autowired
    private ProductService service;

    @GetMapping
    public List<Product> list() {
        return service.getAll();
    }

    @GetMapping("/{id}")
    public Product get(@PathVariable Long id) {
        return service.getById(id);
    }

    @PostMapping
    public Product create(@RequestBody Product product) {
        return service.create(product);
    }

    @PutMapping("/{id}")
    public Product update(@PathVariable Long id, @RequestBody Product product) {
        return service.update(id, product);
    }

    @DeleteMapping("/{id}")
    public void delete(@PathVariable Long id) {
        service.delete(id);
    }
}
```



Paso 7 – CORS global (opcional pero recomendado)

En vez de usar `@CrossOrigin` en cada controller, puedes añadir una config global:

```
// src/main/java/com/example/backend/config/CorsConfig.java
package com.example.backend.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class CorsConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
            .allowedOrigins("http://localhost:3000")
            .allowedMethods("GET", "POST", "PUT", "DELETE")
            .allowedHeaders("*");
    }
}
```

Este patrón está tal cual en la guía 3.2.2 para permitir que el React en 3000 hable con el backend en 8080.



Paso 8 – Swagger

Si decides usar **springdoc-openapi** en vez de springfox (más moderno para Spring Boot 3):

En pom.xml:

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.5.0</version> <!-- o versión estable actual -->
</dependency>
```

Con eso ya tienes la documentación automática en:

- <http://localhost:8080/swagger-ui/index.html>



Paso 9 – Ejecutar y probar el backend

1. Asegúrate que **MySQL está corriendo** y la BD tienda existe.
2. Desde el IDE o consola:
3. `mvn spring-boot:run`
4. Verifica que no hay errores de conexión a la BD.
5. Probar endpoints:
 - Desde Swagger UI (si está configurado).
 - Desde Postman / Thunder Client:
 - GET `http://localhost:8080/api/v1/products`
 - POST `http://localhost:8080/api/v1/products` con cuerpo JSON:

```
{
  "name": "Notebook Gamer",
  "category": "Tecnología",
  "price": 899990
}
```
6. Revisar en MySQL Workbench o Xampp, Laragon, que los datos se están guardando en la tabla `products`.

Con estos 9 pasos el **backend queda listo para que el frontend React consuma la API REST**.

front paso a paso suponiendo que ya tienes el backend con Spring Boot exponiendo:

- GET /api/v1/products
- POST /api/v1/products
- PUT /api/v1/products/{id}
- DELETE /api/v1/products/{id}

y que corre en `http://localhost:8080`.

1. Crear el proyecto React

Si usas CRA:

```
npx create-react-app frontend
cd frontend
npm start
```

(Usa el nombre que quieras en vez de `frontend`.)

```
npm install bootstrap
```

Y en `src/index.js`:

```
import 'bootstrap/dist/css/bootstrap.min.css';
```

2. Definir la URL base de la API

Crea una carpeta `src/services/` y dentro un archivo `productService.js`:

```
// src/services/productService.js
import axios from "axios";

// URL base del backend Spring Boot
const API_BASE = "http://localhost:8080/api/v1/products";

/**
 * Obtiene todos los productos desde el backend (GET)
 */
export async function getProducts() {
  const res = await axios.get(API_BASE);
  return res.data;
}

/**
 * Crea un nuevo producto (POST)
 */
export async function createProduct(product) {
  const res = await axios.post(API_BASE, product);
  return res.data;
}

/**
 * Actualiza un producto existente (PUT)
 */
export async function updateProduct(id, product) {
  const res = await axios.put(`${API_BASE}/${id}`, product);
  return res.data;
}

/**
 * Elimina un producto por su ID (DELETE)
 */
export async function deleteProduct(id) {
  await axios.delete(`${API_BASE}/${id}`);
}
```

Si tienes CORS en el backend, asegúrate de permitir `http://localhost:3000`.

3. Crear un componente para listar productos

Crea `src/components/ProductList.jsx`:

```
import { useEffect, useState } from "react";
import { getProducts, deleteProduct } from "../services/productService";

export default function ProductList({ onEdit }) {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  const load = async () => {
    try {
      setLoading(true);
      const data = await getProducts();
      setProducts(data);
      setError("");
    } catch (err) {
      setError(err.message || "Error cargando productos");
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    load();
  }, []);

  const handleDelete = async (id) => {
    if (!window.confirm("¿Eliminar producto?")) return;
    try {
      await deleteProduct(id);
      setProducts(prev => prev.filter(p => p.id !== id));
    } catch (err) {
      alert("Error al eliminar");
    }
  };

  if (loading) return <p>Cargando...</p>;
  if (error) return <p className="text-danger">{error}</p>;

  return (
    <table className="table table-striped">
      <thead>
        <tr>
          <th>ID</th>
          <th>Nombre</th>
          <th>Precio</th>
          <th>Categoría</th>
          <th></th>
        </tr>
      </thead>
      <tbody>
        {products.map(p => (
          <tr key={p.id}>
            <td>{p.id}</td>
            <td>{p.name}</td>
            <td>{p.price}</td>
            <td>{p.category}</td>
            <td className="text-end">
              <button
                className="btn btn-sm btn-warning me-2"
                onClick={() => onEdit(p)}
              >
                Editar
              </button>
              <button
                className="btn btn-sm btn-danger"
                onClick={() => handleDelete(p.id)}
              >
                Eliminar
              </button>
            </td>
          </tr>
        ))}
      </tbody>
    </table>
  );
}
```

```

        </tr>
      )})
    </tbody>
  </table>
);
}

```

4. Crear un formulario para crear / editar productos

Crea `src/components/ProductForm.jsx`:

```

import { useEffect, useState } from "react";
import { createProduct, updateProduct } from "../services/productService";

// Estado inicial del producto vacío
const emptyProduct = { id: null, name: "", price: "", category: "" };

export default function ProductForm({ selected, onSave, onCancel }) {
  // Estado del formulario y de mensajes de error
  const [product, setProduct] = useState(emptyProduct);
  const [error, setError] = useState("");

  // 📺 useEffect: cada vez que cambia "selected", carga el producto a editar
  useEffect(() => {
    if (selected) setProduct(selected);
    else setProduct(emptyProduct);
  }, [selected]);

  // 🌱 Actualiza el estado al escribir en cualquier input
  const handleChange = (e) => {
    const { name, value } = e.target;
    setProduct((p) => ({ ...p, [name]: value }));
  };

  // 💾 Envía el formulario (crear o actualizar)
  const handleSubmit = async (e) => {
    e.preventDefault();
    setError("");

    // Validación básica
    if (!product.name.trim()) {
      setError("El nombre es obligatorio");
      return;
    }
    if (!product.price || isNaN(product.price)) {
      setError("El precio debe ser numérico");
      return;
    }

    // ✅ El payload debe coincidir con el modelo backend

    const payload = {
      name: product.name.trim(),
      price: Number(product.price),
      category: product.category.trim(),
    };

    try {
      if (product.id == null) {
        // POST → crear producto
        await createProduct(payload);
      } else {
        // PUT → actualizar producto
        await updateProduct(product.id, payload);
      }

      // Avisar al componente padre que se guardó con éxito
      onSave();
      // Limpiar formulario
      setProduct(emptyProduct);
    } catch (err) {
      setError(err.message || "Error al guardar");
    }
  };
}

```

```

    }
  };

  // 🧱 Render del formulario
  return (
    <form onSubmit={handleSubmit} className="mb-4">
      <h4>{product.id == null ? "Nuevo producto" : "Editar producto"}</h4>

      {/* Mensaje de error */}
      {error && <div className="alert alert-danger">{error}</div>}

      {/* Campo: Nombre */}
      <div className="mb-2">
        <label className="form-label">Nombre</label>
        <input
          className="form-control"
          name="name"
          value={product.name}
          onChange={handleChange}
          placeholder="Ej: Notebook"
        />
      </div>

      {/* Campo: Precio */}
      <div className="mb-2">
        <label className="form-label">Precio</label>
        <input
          className="form-control"
          type="number"
          name="price"
          value={product.price}
          onChange={handleChange}
          placeholder="Ej: 899990"
        />
      </div>

      {/* Campo: Categoría */}
      <div className="mb-2">
        <label className="form-label">Categoría</label>
        <input
          className="form-control"
          type="text"
          name="category"
          value={product.category}
          onChange={handleChange}
          placeholder="Ej: Tecnología"
        />
      </div>

      {/* Botones de acción */}
      <button className="btn btn-primary me-2" type="submit">
        Guardar
      </button>
      {selected && (
        <button
          className="btn btn-secondary"
          type="button"
          onClick={onCancel}
        >
          Cancelar
        </button>
      )}
    </form>
  );
}

```

5. Conectar todo en una página **ProductsPage**

Crea `src/pages/ProductsPage.jsx`:

```
import { useState, useRef } from "react";
import ProductList from "../components/ProductList";
import ProductForm from "../components/ProductForm";
import { getProducts } from "../services/productService";

export default function ProductsPage() {
  const [selected, setSelected] = useState(null);
  const [reloadFlag, setReloadFlag] = useState(0);
  const listRef = useRef(null);

  const handleSaved = () => {
    // estrategia simple: forzar recarga de lista vía flag
    setReloadFlag(f => f + 1);
    setSelected(null);
  };

  const handleCancel = () => setSelected(null);

  return (
    <div className="container my-4">
      <h2>Gestión de productos</h2>

      <ProductForm
        selected={selected}
        onSave={handleSaved}
        onCancel={handleCancel}
      />

      {/*
        Una opción simple: pasar reloadFlag y usarlo como dependencia en
        useEffect
        del listado.
      */}
      <ProductList
        key={reloadFlag}
        onEdit={(p) => setSelected(p)}
      />
    </div>
  );
}
```

6. Definir la ruta en **App.jsx**

Si ya usas React Router:

```
npm install react-router-dom
```

En `src/App.jsx`:

```
import { BrowserRouter, Routes, Route, Navigate } from "react-router-dom";
import ProductsPage from "../pages/ProductsPage";
// import Navbar, Footer, etc.

export default function App() {
  return (
    <BrowserRouter>
      {/* <Navbar /> */}
      <Routes>
        <Route path="/" element={<Navigate to="/products" />} />
        <Route path="/products" element={<ProductsPage />} />
        {/* otras rutas... */}
      </Routes>
      {/* <Footer /> */}
    </BrowserRouter>
  );
}
```

7. Probar el flujo completo

1. Levanta el **backend** (Spring Boot) → debería exponer `http://localhost:8080/api/v1/products`.
2. Levanta el **frontend** con `npm start`.
3. Entra a `http://localhost:3000/products`:
 - Verifica que se carguen los productos iniciales (respuesta del `GET /api/v1/products`).
 - Crea un nuevo producto desde el formulario → revisa en la tabla y en la base de datos.
 - Edita un producto existente → verifica actualización.
 - Elimina un producto → verifica que se llame `DELETE /api/v1/products/{id}` y desaparezca de la lista.

Si algo no responde, revisa:

- CORS en el backend.
- La URL base en `productService.js`.
- Errores en la consola del navegador.